Logical Agents, Inference, and Knowledge Representation

Professor Nihad Karim Chowdhury
Department of Computer Science and Engineering
University of Chittagong

Lecture Overview

- **Logical Agents:** Propositional and first-order logic.
- **Inference:** Resolution, forward chaining, backward chaining.
- **Knowledge Representation:** Ontologies, semantic networks, frames.

Lecture Overview

- **Logical Agents:** Propositional and first-order logic.
- **Inference:** Resolution, forward chaining, backward chaining.
- **Knowledge Representation:** Ontologies, semantic networks, frames.

Introduction to Logical Agents

- **What is a Logical Agent?**
  - **Definition:** A logical agent is an intelligent agent that makes decisions based on a logical representation of the world. It uses logical reasoning to infer new knowledge and make decisions on how to act.
  - **Example:** Imagine a robotic vacuum cleaner tasked with cleaning a room. The vacuum can sense the environment (e.g., detect obstacles) and use logical reasoning to navigate around them, ensuring that it does not bump into objects while cleaning efficiently.

Components of a Logical Agent

- **Knowledge Base (KB):** This is the core component where the agent stores facts about the environment. The knowledge base is updated based on new information the agent gathers as it interacts with the environment. The KB is often represented using propositional or first-order logic.
  - **Example:** In the case of the robotic vacuum, the knowledge base might store the fact that there is an obstacle in front of it or that a certain part of the room has already been cleaned.

Components of a Logical Agent

- **Inference Mechanism:** The logical agent uses inference rules (such as modus ponens) to derive new information from the KB. This mechanism allows the agent to reason about the environment and predict the outcomes of its actions.
  - **Example:** If the knowledge base has a fact that "if there is an obstacle, the agent should turn left," and the agent detects an obstacle, it can infer that it should turn left.

Components of a Logical Agent

- **Decision-Making Process:** The agent uses its knowledge and inferred facts to decide on the next action. This decision-making process ensures that the agent chooses the most appropriate action based on its current knowledge and objectives.
  - **Example:** The robotic vacuum decides to clean a different area based on the current state of the room.

Components of a Logical Agent

- **Diagram Suggestion:**

A flowchart of a logical agent:

Inputs (Percepts) → KB → Inference → Decision-Making → Action → Update KB → Repeat.

Propositional vs First Order Logic

- **Key Points:**
  - **Propositional Logic deals** with simple, atomic propositions (statements) and does not allow reasoning about specific objects or their relationships.
  - **First-Order Logic (FOL**) introduces objects, relations, functions, and quantifiers, allowing much more complex and detailed reasoning.
  - FOL is more expressive and flexible, which makes it ideal for scenarios that involve multiple objects and their interactions, such as in AI systems for knowledge representation and reasoning.

Propositional vs First Order Logic

- **Propositional Logic Representation:**
  - In **Propositional Logic**, we deal with simple atomic statements that are either true or false. Here, we cannot specify which ground or where it is raining because propositional logic does not handle objects or relations. It only represents the overall truth of statements.
- Let:
  - R represent **"It is raining"**.
  - W represent **"The ground is wet"**.
- To represent **"It is raining and the ground is wet"**, we use logical conjunction (AND):

$$R \wedge W$$

This means both R (it is raining) and W(the ground is wet) are true.

Propositional vs First Order Logic

- **First-Order Logic (FOL) Representation**

In First-Order Logic, we can introduce objects (like a specific location) and relations (like the ground being wet). We can also specify which ground is wet or where it is raining.

- Define:
  - **Objects:** Let x be an object representing a location (such as a specific ground area).
  - **Predicates**:
    - Raining(x) represents **"It is raining at location x".**
    - Wet(y) represents **"Object y is wet"** (e.g., the ground being wet).
- To represent **"It is raining and the ground is wet"** in FOL:
  - Raining(x)∧Wet(Ground) This means it is raining at location x, and the ground is wet at that location.

Propositional vs First Order Logic

- **Further Generalization (optional):** If you want to generalize the statement for all locations, you could use the universal quantifier ∀:

$$\forall x(\text{Raining}(x) \Rightarrow \text{Wet}(\text{Ground}(x)))$$

This means that for every location x, if it is raining at that location, then the ground at that location is wet.

Propositional vs First Order Logic

- **Summary of Representations:**
  - **Propositional Logic:**

    $R \wedge W$ (It is raining and the ground is wet, without specifying location or details.)
  - **First-Order Logic (FOL):**

    Raining(x)∧Wet(Ground(x)) (It is raining at a specific location, and the ground at that location is wet.) FOL allows for more detailed representation by introducing objects (like locations) and their relationships, which propositional logic cannot handle.

Logical Agent (Cleaning Robot Example)

- **Components of a Logical Agent**
  - **Knowledge Base (KB):** The knowledge base contains facts the robot knows about the environment. For example:
    - **Obstacle(x, y):** There is an obstacle at position (x, y).
    - **Dirt(x, y):** There is dirt at position (x, y).
    - **Clean(x, y):** The robot cleaned position (x, y).

Logical Agent (Cleaning Robot Example)

- **Components of a Logical Agent**
  - **Inference Mechanism:** The robot uses logical rules to infer new knowledge based on its current observations and KB. For example:
    - **If Dirt(x, y), then Clean(x, y).** (If there is dirt at a position, the robot should clean that position.)
    - **If Obstacle(x, y), then Avoid(x, y).** (If there is an obstacle, the robot should avoid that position.)

Logical Agent (Cleaning Robot Example)

- **Components of a Logical Agent**
  - **Decision-Making:** Based on the knowledge base and inferred information, the robot makes decisions about where to move and what actions to take. For example:
    - **If Dirt(x, y) ∧ ¬Obstacle(x, y), then Move(x, y).** (If there is dirt at a position and no obstacle, move to that position.)

Cleaning Robot Example in **Propositional Logic**

- **Initial Scenario:**
  - The robot is in a 3x3 grid room.
  - Dirt is detected at position (1,2), and obstacles are at positions (2,2) and (3,3).
- **Propositions for the Robot:**
  - D(1,2): There is dirt at position (1,2).
  - O(2,2): There is an obstacle at position (2,2).
  - O(3,3): There is an obstacle at position (3,3).
  - R(x,y): The robot is at position (x, y).

Cleaning Robot Example in **Propositional Logic**

- **Logical Rules:**
  - If the robot detects dirt at a position, it should clean it: **D(x,y)⇒Clean(x,y)**
  - If there is an obstacle, the robot should avoid it: **O(x,y)⇒¬Move(x,y)**
  - If the robot has cleaned a position, it updates its KB to reflect the new state: **Clean(x,y)⇒¬D(x,y)**

- **Goal:** The robot must clean the dirt in the room without hitting any obstacles.

Cleaning Robot Example in **Propositional Logic**

- **Propositional Logic for Robot's Actions**

– Step-by-Step Example:

  - The robot starts at **R(1,1).**
  - Using sensors, it detects **dirt** at **(1,2)**: D**(1,2)**
  - It uses propositional logic to decide its next move:
    - There is no obstacle between **R(1,1)** and **R(1,2)**, so the rule O**(1,2)⇒¬Move(1,2)** is false (meaning it's safe to move).
  - Based on its KB and the rule D**(1,2)⇒Clean(1,2)**, the robot moves to **(1,2)** and cleans the dirt.

Cleaning Robot Example in **Propositional Logic**

- **Inference Process:**
  - The robot updates its knowledge base after cleaning: **Clean(1,2)⇒¬D(1,2)** (There is no longer dirt at position **(1,2)**.)
- **Next Action:**
  - The robot detects an obstacle at **(2,2)**: O**(2,2).**
  - It infers that moving to **(2,2)** is not possible, so it looks for an alternative path to clean the room.

Cleaning Robot Example in **FOL**

- **Example (Cleaning Robot):** Imagine a robotic vacuum tasked with cleaning rooms in a house. The robot can reason about **objects** (e.g., rooms, dirt, obstacles) and their **relations** (e.g., "is in", "is adjacent to"). It can also reason about the outcomes of its actions, like moving or cleaning.
- **Difference from Propositional Logic:** FOL allows the robot to handle variables and quantifiers, which helps in generalizing rules (e.g., "For all rooms, if there is dirt, clean the room.") instead of dealing with specific statements for every room.

Cleaning Robot Example in **FOL**

- **Knowledge Base (KB):** The robot's knowledge base consists of facts about the environment in the form of objects, their properties, and relations.
  - **Objects:** Specific entities like rooms, dirt patches, and obstacles.
  - **Relations:** Relationships between objects, such as **Adjacent(room1, room2**).
  - **Example:** The robot might store the following facts in its KB:
    - **Room(r1), Dirt(r1), Obstacle(r2)** – There is dirt in room 1, and an obstacle in room 2.
    - **Adjacent(r1, r2)** – Room 1 is adjacent to room 2.

Cleaning Robot Example in **FOL**

- **Inference Mechanism:** The robot uses logical rules to deduce new information from its knowledge base. With FOL, the robot can make more generalized inferences, such as **"If a room has dirt, clean it, unless an obstacle prevents movement to that room."**
  - Example Rule:

    $\forall x(\text{Room}(x) \land \text{Dirt}(x) \Rightarrow \text{Clean}(x))$

    (For every room, if there is dirt in that room, then clean it.)

Cleaning Robot Example in **FOL**

- **Decision-Making Process:** The robot decides what actions to take (e.g., moving or cleaning) based on its current knowledge and inferred rules.
  - Example: **If the robot knows it is in room 1, and there is dirt in the room, it decides to clean the room.**

Cleaning Robot Example in **FOL**

Step by Step Example:

- **Perception:** The robot perceives that it is in room r1, where there is dirt, and that room r2 has an obstacle. It also knows room r3 is clean.
- **Initial Knowledge:**
  - Room(r1), Dirt(r1), At(robot, r1)
  - Room(r2), Obstacle(r2)
  - Room(r3), Clean(r3)
  - Adjacent(r1, r2), Adjacent(r2, r3)

Cleaning Robot Example in **FOL**

Step by Step Example:

- **Inference (using General Rules):**
  - From the general rule: $\forall x(\text{Room}(x) \wedge \text{Dirt}(x) \Rightarrow \text{Clean}(x))$ the robot deduces that it should clean room r1.
  - After cleaning r1, the robot updates its KB: $\neg\text{Dirt}(r1), \text{Clean}(r1)$
  - Next, the robot considers moving to an adjacent room. It applies the rule: $\forall x,y(\text{Adjacent}(x,y) \wedge \neg\text{Obstacle}(y) \Rightarrow \text{Move}(x,y))$ It infers that it cannot move to r2 because of the obstacle.
  - The robot then considers moving to r3 (since **Adjacent(r1, r2)** and there is no obstacle in r3).
- **Result:** The robot moves from r1 to r3 and updates its KB.

Propositional vs FOL (More Examples)

- **Computer Science Student can do programming well**
  - **Propositional Logic:** In Propositional Logic, we represent the whole statement as a single proposition since we cannot handle specific objects or relationships between them.
  - Let:
    - S represent **"The person is a Computer Science Student".**
    - P represent **"The person can do programming well".**
  - The sentence "Computer Science Student can do programming well" can be represented as:

    $$S \Rightarrow P$$

    This means if the person is a Computer Science student, then they can do programming well.

Propositional vs FOL (More Examples)

- **Computer Science Student can do programming well**
  - **First-Order Logic (FOL):** In FOL, we can express specific relations between objects like students and their abilities.
  - Define:
    - **CSStudent(x):** x is a Computer Science student.
    - **CanProgramWell(x):** x can program well.
  - The sentence "Computer Science Student can do programming well" can be represented as:

$$\forall x(\text{CSStudent}(x) \Rightarrow \text{CanProgramWell}(x))$$

This means for all x, if x is a Computer Science student, then x can program well.

Propositional vs FOL (More Examples)

- **Man is mortal**
  - **Propositional Logic:** In propositional logic, we can't express the concept of "man" as a class of individuals, so we simplify it to:
  - Define:
    - M represent **"Man"**.
    - T represent **"Mortal"**.
  - The sentence "Man is mortal" can be represented as:

$$\mathbf{M \Rightarrow T}$$

This means if someone is a man, then they are mortal.

Propositional vs FOL (More Examples)

- **Man is mortal**
  - **First-Order Logic (FOL):**
  - Define:
    - **Man(x):** x is a man.
    - **Mortal(x):** x is mortal.
  - The sentence **"Man is mortal"** can be represented as:

$$\forall x(Man(x) \Rightarrow Mortal(x))$$

This means for all x, if x is a man, then x is mortal.

Propositional vs FOL (More Examples)

- **Cough turns to fever**
  - **Propositional Logic:**
  - Define:
    - **C** represent "The person has a cough".
    - **F** represent "The person develops a fever".
  - The sentence "Cough turns to fever" can be represented as:

$$\mathbf{C \Rightarrow F}$$

    This means if a person has a cough, then they will develop a fever.

Propositional vs FOL (More Examples)

- **Cough turns to fever**
  - **First-Order Logic (FOL):**
  - Define:
    - **Cough(x):** x has a cough.
    - **Fever(x):** x has a fever.
  - The sentence "Cough turns to fever" can be represented as:

$$\forall x(\text{Cough}(x) \Rightarrow \text{Fever}(x))$$

This means for all x, if x has a cough, then x will develop a fever.

Lecture Overview

- **Logical Agents:** Propositional and first-order logic.
- **Inference:** Resolution, forward chaining, backward chaining.
- **Knowledge Representation:** Ontologies, semantic networks, frames.

Inference

- **Inference** is the process of drawing conclusions or deducing new information based on known facts or premises. In logic and artificial intelligence, inference involves applying logical rules or operations to a set of given statements (facts, rules, or observations) to derive new statements or conclusions that logically follow from them. In simpler terms, inference allows us to:
  - Start with known information (facts or premises).
  - Apply logical reasoning or rules.
  - Arrive at new conclusions that were not explicitly stated but can be logically derived.

Inference

Involves drawing conclusions that are necessarily true if the premises are true.

- Example:
  - **Premise 1: "All humans are mortal.“**
  - **Premise 2: "Socrates is a human.“**
  - **Conclusion: "Socrates is mortal.“**

The conclusion is a guaranteed truth if the premises are correct.

Inference in Logic:

In logic (such as propositional logic or first-order logic), inference involves deriving new facts from a set of known facts using inference rules like modus ponens, resolution, or unification.

Modus Ponens: If you know that "If A, then B" and "A is true," you can infer that "B is true.“

Example:

Premise 1: "If it is raining, the ground will be wet.“

Premise 2: "It is raining.“

Conclusion: "The ground is wet."

Inference Using Propositional Logic (Cleaning Robot Example)

In Propositional Logic, the robot uses simple propositions (statements that are either true or false) to represent the state of the environment and to infer what actions to take.

Inference Using Propositional Logic (Cleaning Robot Example)

Environment:

The cleaning robot operates in a 2x2 room grid.

The robot can either clean or move, and it knows the location of dirt and obstacles.

We define the following propositions:

R**(xy):** The robot is in position (x, y).

D**(xy):** There is dirt in position (x, y).

C**(xy):** The robot can clean position (x, y).

O**(xy):** There is an obstacle in position (x, y).

M**(xy):** The robot can move to position (x, y).

Inference Using Propositional Logic (Cleaning Robot Example)

- **Given Knowledge:**
  - The robot starts at position (1,1): **R(11)** .
  - There is dirt in position (1,2): **D(12)** .
  - There is an obstacle in position (2,1): **O(21)**.

Inference Using Propositional Logic (Cleaning Robot Example)

- **Logical Rules:**

  1. If there is dirt in the current position, clean it:

$$R(\mathbf{xy}) \wedge D(\mathbf{xy}) \Rightarrow C(\mathbf{xy})$$

(If the robot is in position x,y and there is dirt in x,y, the robot can clean

that position.)

2. If there is no obstacle, the robot can move to an adjacent position:

$$\mathbf{R(xy) \wedge \neg O(ab) \Rightarrow M(ab)}$$

(If the robot is in position x,y and there is no obstacle in position a,b, the robot can move to position a,b.)

Inference Using Propositional Logic (Cleaning Robot Example)

Inference Example:

- **Starting point:** The robot is at position (1,1): $R(11)$.
- **Dirt detection:** The robot knows there is dirt at $D(12)$ (position (1,2)).
- **Inference 1 (Moving):** Using the rule $\boldsymbol{R(11) \wedge \neg O(12) \Rightarrow M(12)}$, the robot infers that it can move to $M12$ because there is no obstacle at position (1,2).
- **Action 1:** The robot moves to $M12$.
- **Inference 2 (Cleaning):** Once the robot is at position (1,2), it checks the rule $\boldsymbol{R(12) \wedge D(12) \Rightarrow C(12)}$. Since there is dirt at (1,2), the robot infers that it should clean that position.
- **Action 2:** The robot cleans $C(12)$, and the dirt is removed.

This shows how the robot uses propositional logic to make inferences about movement and cleaning actions.

Inference Using FOL (Cleaning Robot Example)

In First-Order Logic (FOL), the robot uses objects, predicates, and quantifiers to reason about its environment, enabling more complex and general inferences.

Inference Using FOL (Cleaning Robot Example)

Environment:

- The robot operates in a 2x2 grid with rooms represented as objects.
- The robot can sense obstacles, dirt, and its location, and it can either move or clean.
- We define:
  - Objects: r1,r2 represent rooms (positions) in the grid.
  - Predicates:
    - **RobotAt(x):** The robot is at room x.
    - **Dirt(x):** There is dirt in room x.
    - **Obstacle(x):** There is an obstacle in room x.
    - **Clean(x):** The robot can clean room x.
    - **Move(x,y):** The robot can move from room x to room y.

Inference Using FOL (Cleaning Robot Example)

Given Knowledge:

- The robot starts in room r1: **RobotAt(r1)**.
- There is dirt in room r2: **Dirt(r2)**.
- There is no obstacle in room r2: **¬Obstacle(r2)**

Inference Using FOL (Cleaning Robot Example)

Logical Rules (FOL):

- If the robot is in a room and there is dirt in that room, clean it:

  $\forall x(\text{RobotAt}(x) \wedge \text{Dirt}(x) \Rightarrow \text{Clean}(x))$ (For all rooms x, if the robot is at x and there is dirt at x, then the robot should clean x.)

- If there is no obstacle in the next room, the robot can move to it:

  $\forall x,y(\text{RobotAt}(x) \wedge \neg\text{Obstacle}(y) \Rightarrow \text{Move}(x,y))$ (For all rooms x and y, if the robot is at x and there is no obstacle in y, it can move from x to y.)

Inference Using FOL (Cleaning Robot Example)

Inference Example:

- **Starting point:** The robot is at room r1: **RobotAt(r1).**
- **Dirt detection:** The robot knows there is dirt in room r2: **Dirt(r2)**.
- **Inference 1 (Movement):** Using the rule $\forall x,y(\text{RobotAt}(x) \wedge \neg \text{Obstacle}(y) \Rightarrow \text{Move}(x,y))$, the robot infers that it can move to r2 since there is no obstacle in r2.
- **Action 1:** The robot moves from r1 to r2, and the new state is **RobotAt(r2)**.
- **Inference 2 (Cleaning):** Using the rule $\forall x(\text{RobotAt}(x) \wedge \text{Dirt}(x) \Rightarrow \text{Clean}(x))$, the robot infers that it can clean room r2 because there is dirt in room r2 and the robot is now in r2.
- **Action 2:** The robot cleans r2, and the dirt is removed.

What is Resolution?

- **Resolution** is a rule of inference used in **propositional logic or first-order logic** to derive new conclusions from a set of statements (also called clauses). It is particularly useful in automated theorem proving, where the goal is to prove or disprove a conclusion based on a set of given premises.

- The **key idea** of resolution is that if you have two clauses:
  - One containing a proposition (e.g., A)
  - Another containing the **negation** of that proposition (e.g.,¬A)

You can **resolve** them by combining the remaining parts of both clauses into a new clause that excludes A and ¬A.

Basic Example of Resolution

- **Problem:**

Suppose we have the following two logical statements (clauses):

1. **AVB** (Either A or B is true)
2. **¬AVC** (Either A is false, or C is true)

We want to use **resolution** to infer something new from these statements.

Step-by-Step Resolution Process

Step 1: Look for Opposing Literals

We are trying to find a literal (a proposition) in one clause and its negation in the other clause. In this case:

- The first clause contains **A** (positive literal).
- The second clause contains **¬A** (the negation of **A**).

Since **A** and **¬A** oppose each other, we can apply the resolution rule.

Step-by-Step Resolution Process

Step 2: Eliminate the Opposing Literals

The resolution rule says that if a literal and its negation appear in different clauses, you can eliminate both of them and combine the remaining parts of the clauses.

Here's how that works:

- From the first clause **A∨B**, remove **A** (the literal being resolved).
- From the second clause **¬A∨C**, remove **¬A** (the negation being resolved).

Now we are left with:

- **B** from the first clause
- **C** from the second clause

The new clause formed by combining the remaining parts is: **B∨C**

This means that either **B** or **C** must be true, based on the original statements.

Step-by-Step Resolution Process

Step 3: Repeat if Necessary

We can continue applying the resolution rule on other pairs of clauses until we derive the conclusion we are trying to prove.

Example with Proof by Contradiction

Let's now apply **resolution** to solve a problem using **proof by contradiction**.

Problem:

We are given the following premises:

A∨B (Either **A** or **B** is true)

¬A∨C (Either **A** is false or **C** is true)

¬B (**B** is false)

We want to prove that **C** is true.

Example with Proof by Contradiction

- **Step 1: Negate the Conclusion**

To use resolution to prove **C**, we first **negate** the conclusion and add it as a new clause:

¬C(assume C is false)

Now, the set of clauses is:

1. AVB

2. ¬AVC

3. ¬B

4. ¬C

Example with Proof by Contradiction

- **Step 2: Apply Resolution**

We now apply the resolution rule to derive a contradiction.

- **Resolution Step 1**: Resolve **¬AVC** and **¬C**.
  These two clauses contain **C** and **¬C**. By resolving them, we get:

  ¬A (The new clause is **¬A**).

- **Resolution Step 2**: Resolve **AVB** and **¬A**.
  These two clauses contain **A** and **¬A**. By resolving them, we get:

  B (The new clause is **B**).

- **Resolution Step 3**: Resolve **B** and **¬B**.
  These two clauses contain **B** and **¬B**. By resolving them, we get an **empty clause**: □ (The empty clause represents a contradiction).

Example with Proof by Contradiction

- **Step 3: Conclusion**

Since we derived a contradiction, the assumption **¬C** must be false, meaning **C** must be true. Therefore, we have proven that **C** is true.

Example with Proof by Contradiction

- **Recap of Key Concepts**
  - **Literal:** A proposition (e.g., **A**) or its negation (e.g., **¬A**).
  - **Clause:** A disjunction of literals (e.g., **AVB**).
  - **Resolution Rule:** If two clauses contain a literal and its negation, we can resolve them by removing the opposing literals and combining the remaining parts.
  - **Proof by Contradiction:** To prove a statement **C**, assume **¬C** and apply resolution until we derive a contradiction, proving **C** is true.

Resolution (Cleaning Robot Problem)

- **Knowledge Base (KB):**
- **Clause 1:** “Robot is not in the room or there is no dirt, or the robot does not need to clean the room. “

  ¬RobotInRoom ∨ ¬Dirt ∨ ¬Clean
- **Clause 2:** "The robot is in the room.“

  RobotInRoom
- **Clause 3:** "There is dirt in the room.“

  Dirt

Goal: The robot should clean the room, **Clean**, using resolution by contradiction.

Resolution (Cleaning Robot Problem)

- **Step 1: Negate the Goal**

We want to prove **Clean**, so we negate the goal and add **¬Clean** to the knowledge base.

Negated Goal: ¬Clean

Resolution (Cleaning Robot Problem)

- **Step 2: Apply Resolution**

We now resolve the clauses from the knowledge base with the negated goal **¬Clean** until we reach an empty clause or find a contradiction.

- **Resolution Step 1:**

Resolve Clause 1 (**¬RobotInRoomV¬DirtV¬Clean**) with Clause 2 **RobotInRoom**.

- **¬RobotInRoomV¬DirtV¬Clean**
- **RobotInRoom**

When we resolve on **RobotInRoom** and **¬RobotInRoom**, we eliminate those terms and get the result: **¬Dirt V ¬Clean**

Resolution (Cleaning Robot Problem)

- **Resolution Step 2:**

Resolve **¬Dirt V ¬Clean** with Clause 3 **Dirt**.

- **¬Dirt V ¬Clean**
- **Dirt**

When we resolve on **Dirt** and **¬Dirt**, we eliminate those terms and get the result: **¬Clean**

Resolution (Cleaning Robot Problem)

- **Resolution Step 3:**

Finally, we resolve **¬Clean** (the result from the previous step) with the negated goal **¬Clean**.

- **¬Clean**
- **¬Clean**

When we resolve **¬Clean** with itself, we get an empty clause.

Resolution (Cleaning Robot Problem)

- **Step 3: Interpret the Result (Empty Clause)**

By reaching an empty clause, we have found a contradiction. This means that our assumption **¬Clean** is inconsistent with the knowledge base. Thus, the original goal **Clean** must be true.

Forward Chaining in Propositional Logic

What is Forward Chaining?

Forward chaining is **a data-driven inference technique** used in logic and AI, where the system starts with known facts (in the knowledge base) and applies inference rules to deduce new facts. This process continues until it deduces the goal, or no more new facts can be inferred.

In **Propositional Logic**, forward chaining uses a set of **propositional statements** and **implication rules** to infer new facts from existing ones.

Forward Chaining in Propositional Logic

Step-by-Step Algorithm for Forward Chaining:

Here's the step-by-step algorithm for forward chaining:

- **Initialize the Knowledge Base (KB):** Start with a set of known facts or propositions.
- **Identify Inference Rules:** Define rules in the form of implications, such as $P1 \wedge P2 \Rightarrow P3$, where $P1$ and $P2$ are premises, and $P3$ is the conclusion.
- **Find Satisfied Rules:** Check all rules to see if the premises (left-hand side) are satisfied by the facts in the knowledge base.

Forward Chaining in Propositional Logic

Step-by-Step Algorithm for Forward Chaining:

- **Infer New Facts:** If a rule is satisfied (all premises are true), then infer the conclusion (right-hand side) and add it to the knowledge base.
- **Repeat the Process:** Continue checking the rules and inferring new facts until no more facts can be inferred, or the goal is reached.
- **Stop Condition:** The process stops when either the goal has been reached or no more new facts can be inferred.

Deduction from a Knowledge Base (Cleaning Robot Example)

Let us apply forward chaining to the cleaning robot in a room with dirt and obstacles.

Knowledge Base (KB): Here are the known facts (propositions) in the robot's environment:

$\boldsymbol{R11}$**:** The robot is in room (1,1).

$\boldsymbol{D12}$**:** There is dirt in room (1,2).

$\boldsymbol{O21}$**:** There is an obstacle in room (2,1).

Deduction from a Knowledge Base (Cleaning Robot Example)

- **Inference Rules:**
  - **Cleaning Rule:** If the robot is in a room with dirt, it can clean the room: $\boldsymbol{R(xy) \wedge D(xy) \Rightarrow C(xy)}$ (If the robot is in position $\boldsymbol{x,y}$ and there is dirt in $\boldsymbol{x,y}$, the robot can clean it.)
  - **Movement Rule:** If the robot is in one room, and there is no obstacle in the adjacent room, the robot can move there:$\boldsymbol{R(xy) \wedge \neg O(ab) \Rightarrow M(ab)}$ (If the robot is in position x,y and there is no obstacle in $\boldsymbol{a,b}$, it can move to $\boldsymbol{a,b}$.)

Deduction from a Knowledge Base (Cleaning Robot Example)

Forward Chaining Process (Step-by-Step Deduction):

- **Initial Facts (KB):**
  - R**(11):** The robot is at (1,1).
  - D**(12):** There is dirt at (1,2).
  - O**(21):** There is an obstacle at (2,1).

Deduction from a Knowledge Base (Cleaning Robot Example)

- **Check Inference Rule 1 (Cleaning Rule):**
  - The premises for the cleaning rule $R(xy) \wedge D(xy) \Rightarrow C(xy)$ are:
    - $R(xy)$**:** The robot is in a room.
    - $D(xy)$**:** There is dirt in that room.
  - The robot is at $R(11)$ and there is dirt at $D(12)$, so this rule does not apply yet because the robot is not in room **(1,2)**.
  - No cleaning can happen yet.

Deduction from a Knowledge Base (Cleaning Robot Example)

- **Check Inference Rule 2 (Movement Rule):**
  - The premises for the movement rule $\boldsymbol{R(xy) \wedge \neg O(ab) \Rightarrow M(ab)}$ are:
    - $\boldsymbol{R(xy)}$**:** The robot is in a room.
    - $\boldsymbol{\neg O(ab)}$**:** There is no obstacle in the adjacent room.
  - The robot is at $\boldsymbol{R(11)}$, and there is no obstacle in room **(1,2)** (since the obstacle is in room **(2,1)**).
  - Therefore, the rule applies, and the robot can move from room **(1,1)** to room **(1,2)**.
  - New Fact: $\boldsymbol{R(12)}$ (The robot moves to room **(1,2)**).

Deduction from a Knowledge Base (Cleaning Robot Example)

- **Update the Knowledge Base:**
  - The knowledge base is updated to include the new fact:
    - $\boldsymbol{R(12)}$(The robot is in room **(1,2)**).

Deduction from a Knowledge Base (Cleaning Robot Example)

- **Apply Inference Rule 1 (Cleaning Rule) Again:**
  - Now, the robot is in room **(1,2)**, and there is dirt in this room **($R12 \wedge D12 \Rightarrow C12$)**.
  - Therefore, the cleaning rule applies.
  - New Fact: **$C12$** (The robot cleans room **(1,2)**).
- **Final Knowledge Base:**
  - **$C12$** (Room **(1,2)** is now clean).

Deduction from a Knowledge Base (Cleaning Robot Example)

- The robot used forward chaining to move from room **(1,1)** to room **(1,2)**, inferred that it could clean the room, and performed the cleaning action.
- The process stopped when there were no more rules to apply, and all actions were completed.

Backward Chaining in Propositional Logic

What is Backward Chaining?

Backward chaining is **a goal-driven inference technique** used in logic, where you start with a goal (something you want to prove or achieve) and work backwards to find the facts or conditions that satisfy that goal. In backward chaining, you ask, "What must be true for this goal to be true?" and recursively find supporting facts until you reach known facts or deduce the conclusion.

In **Propositional Logic**, backward chaining applies a set of rules and works backward from the goal to infer the conditions or facts that need to be true.

Backward Chaining in Propositional Logic

Step-by-Step Algorithm for Backward Chaining:

- **Identify the Goal:**
  - Begin with the desired conclusion or goal. This is the fact or proposition you want to prove or achieve.
- **Find Relevant Rules:**
  - Search the rules in your knowledge base to find any rules that have the goal as their conclusion (the "then" part of an "if-then" rule).
- **Sub-goals from Premises:**
  - Once a relevant rule is found, check the premises (the "if" part of the rule). These premises become sub-goals. Each premise must be proven true for the rule's conclusion to hold.

Backward Chaining in Propositional Logic

- **Prove Sub-goals Recursively:**
  - For each sub-goal, apply backward chaining recursively by treating the sub-goal as the new goal and finding rules or facts that support it.
- **Check Known Facts:**
  - If a sub-goal matches a fact in the knowledge base, it is considered proven. If all sub-goals are proven, the original goal is considered achieved.
- **Repeat the Process:**
  - Continue this process until either the goal is proven (all sub-goals are satisfied) or you reach a point where the goal cannot be proven.

Goal-driven Deduction with a Cleaning Robot

- **Knowledge Base (Facts and Rules):**
  - **Facts:**
    - **RobotAt(r1):** The robot is in room r1.
    - **Dirt(r2):** There is dirt in room r2.
    - **Obstacle(r3):** There is an obstacle in room r3.

Goal-driven Deduction with a Cleaning Robot

- **Knowledge Base (Facts and Rules):**
  - **Facts:**
    - **RobotAt(r1):** The robot is in room r1.
    - **Dirt(r2):** There is dirt in room r2.
    - **Obstacle(r3):** There is an obstacle in room r3.
  - **Rules:**
    - **RobotAt(x)∧Dirt(x)⇒Clean(x)**(If the robot is in room x and there is dirt in x, the robot can clean the room.)
    - **RobotAt(x)∧¬Obstacle(y)⇒Move(x,y)**(If the robot is in room x and there is no obstacle in room y, the robot can move to room y.)

Goal-driven Deduction with a Cleaning Robot

Goal:

The goal is for the robot to clean room r2, i.e., Clean(r2).

Goal-driven Deduction with a Cleaning Robot

Backward Chaining Process (Step-by-Step):

Step 1: Start with the Goal

- **Goal: Clean(r2)**.
- The backward chaining process begins by asking, "What conditions must be true for the robot to clean room r2?“

Step 2: Find Relevant Rules

- Look for rules where **Clean(x)** is the conclusion.
- The relevant rule is: **$\text{RobotAt}(x) \wedge \text{Dirt}(x) \Rightarrow \text{Clean}(x)$**

Goal-driven Deduction with a Cleaning Robot

Step 3: Generate Sub-goals from Premises:

- According to the rule, for **Clean(r2)** to be true, the following sub-goals must also be true:
  - **Sub-goal 1: RobotAt(r2)** (The robot must be in room r2).
  - **Sub-goal 2: Dirt(r2)** (There must be dirt in room r2).

Goal-driven Deduction with a Cleaning Robot

Step 4: Prove Sub-goals

Sub-goal 1: RobotAt(r2)

- The robot is currently at **RobotAt(r1)** (room r1), so it's not yet in r2.
- To prove this sub-goal, the robot needs to move to room r2.

Find a Rule for Moving:

- We search for rules where **Move(x,r2)** is the conclusion.
- The relevant rule is: **RobotAt(x)∧¬Obstacle(r2)⇒Move(x,r2)**

Goal-driven Deduction with a Cleaning Robot

Generate New Sub-goals for Moving to r2:

For the robot to move to room r2, two new sub-goals must be satisfied:

- **Sub-goal 1.1: RobotAt(r1)** (The robot is in room r1).
- **Sub-goal 1.2: ¬Obstacle(r2)** (There is no obstacle in room r2).

Goal-driven Deduction with a Cleaning Robot

Prove Sub-goal 1.1:

- **Fact check: RobotAt(r1)** is a known fact in the knowledge base.
- **Conclusion:** Sub-goal 1.1 is proven.

Prove Sub-goal 1.2:

- **Fact check:** There is no known fact in the knowledge base indicating an obstacle in room r2 (the fact is about an obstacle in r3).
- **Conclusion:** Sub-goal 1.2 is proven.
- Since both sub-goals 1.1 and 1.2 are proven, the robot can move to room r2 (Sub-goal 1 is satisfied).

Goal-driven Deduction with a Cleaning Robot

Sub-goal 2: Dirt(r2)

- **Fact check: Dirt(r2)** is a known fact in the knowledge base.
- **Conclusion:** Sub-goal 2 is proven.

Step 5: Achieve the Goal

- Since both Sub-goal 1 (the robot is in room r2) and Sub-goal 2 (there is dirt in room r2) are proven, the robot can now clean room r2.
- The original goal **Clean(r2)** is achieved.

Goal-driven Deduction with a Cleaning Robot

Summary of Backward Chaining:

Goal-Driven Approach: In backward chaining, the process starts from the goal **Clean(r2)** and works backward to find the conditions (sub-goals) that must be satisfied to achieve that goal.

Recursive Proof: Each sub-goal is treated as a new goal, and backward chaining is applied recursively until the sub-goals are either proven by facts or further rules.

Fact Verification: When sub-goals reach a fact in the knowledge base (like **Dirt(r2)** or **RobotAt(r1)**), they are considered proven, and the backward chaining process can proceed.

Efficient Problem-Solving: Backward chaining is particularly efficient for problem-solving in scenarios where the goal is known, and the focus is on proving the conditions necessary to achieve that goal.

Self Study

- **Knowledge Representation: Ontology**

Book Reference

Stuart Russell and Peter Norvig: “Artificial Intelligence a Modern Approach”, 3rd Edition, Pearson Education Asia. (Text Book) (Online) (Lectures) Chapter 7,8,9,12.